

```

#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
#include <numeric>
using namespace std;
using namespace __gnu_pbds;

#define fastio ios::sync_with_stdio(false); cin.tie(nullptr);
#define ll long long
#define sort_(v) sort(all(v), greater<>())
#define vi vector<int>
#define vll vector<ll>
#define vvl vector<vll>
#define vb vector<bool>
#define all(x) (x).begin(), (x).end()
#define pb push_back
#define ff first
#define ss second
#define endl '\n'

const ll mod = 1e9 + 7;
const ll MOD=998'244'353;
const int MAXN=200005;

//-----Bit_Manipulation-----*/
// __builtin_popcountll(num);
//ll msb = 63 - __builtin_clzll(n);
//ll d = __builtin_ctzll(n); power of 2
//string s = "10101"; ll x = stoll(s, nullptr, 2); cout << x; // 21
//int x = 13; string s = bitset<32>(x).to_string();
//Removing trailing zero: s = s.substr(s.find('1'));

/*-----Multiset-----*/
// auto mx = *s.rbegin();
// auto mn = *s.begin();

/*-----SIEVE-----*/
int spf[MAXN];
void sieve(){iota(spf,spf+MAXN,0);for(int i=2;i*i<MAXN;i++){if(spf[i]==i){for(int
j=i*i;j<MAXN;j+=i)if(spf[j]==j) spf[j]=i;}}}
vll get_fac(ll x){vll fac;while(x>1){ll p=spf[x];fac.pb(p);while(x%p==0) x/=p;}return fac;}

/* ---- Dense Utilities ---- */
ll gcd_ll(ll a,ll b){return __gcd(a,b);}
ll gcd_arr(const vll& nums){ll g=0;for(ll x:nums)g=__gcd(g,x);return g;}
ll lcm_ll(ll a,ll b){return a/gcd_ll(a,b)*b;}
bool isPrime(int x){if(x<2) return 0; for(int i=2;i*i<=x;i++) if(x%i==0) return 0; return 1;}
bool isPalindrome(string s){string r=s; reverse(all(r)); return s==r;}
string toUpperCase(string s){transform(all(s),s.begin(),::toupper); return s;}
string toLowerCase(string s){transform(all(s),s.begin(),::tolower); return s;}

```

```

/* ---- Modular ---- */
ll power(ll x, ll y) { ll r = 1; for(x %= MOD; y >= 1; x = x * x % MOD) if(y & 1) r = r * x % MOD; return r; }

/* ---- nCr ---- */
vll fact(MAXN + 1), invfact(MAXN + 1);
void init_ncr() { fact[0] = 1; for(int i = 1; i <= MAXN; i++) fact[i] = fact[i - 1] * i % MOD;
    invfact[MAXN] = power(fact[MAXN], MOD - 2);
    for(int i = MAXN; i > 0; i--) invfact[i - 1] = invfact[i] * i % MOD; }
ll mod_ncr(int n, int r) { return (r < 0 || r > n) ? 0 : fact[n] * invfact[r] % MOD * invfact[n - r] % MOD; }

/* ---- TO_STRING ---- */
string toBin(string n) { if(n == "0") return "0"; string b = ""; while(n != "0") {
    string t = ""; int c = 0; for(char x : n) { int v = c * 10 + (x - '0'); t += char(v / 2 + '0'); c = v % 2; } b += char(c + '0');
    n = t.substr(t.find_first_not_of('0')); if(n == "") n = "0"; } reverse(b.begin(), b.end()); return b; }

/* ---- POW ---- */
long long modpow(long long b, long long e, long long m) {
    long long r = 1; b %= m; while(e) { if(e & 1) r = r * b % m; b = b * b % m; e >>= 1; }
    return r; }
//----Fermat Mod Inverse (using your modpow)----//
ll modinv(ll a) { return modpow(a, MOD - 2, MOD); }
// (a / b) % MOD
ll ans = a * modinv(b) % MOD;

/* ---- PERM ---- */
vll perm_arr(vll a) { sort(all(a)); vll perms;
do { perms.push_back(a); } while (next_permutation(all(a))); return perms; }

vll perm_str(string s) { vll perms; sort(all(s)); do { perms.pb(stoll(s)); } while
(next_permutation(all(s))); return perms; }

/* ---- BS ---- */
ll first_occurrence(const vll& a, ll l, ll r, ll x) { ll ans = -1; for(; l <= r; ) { ll m = (l + r) >> 1; if(a[m] >= x) { if(a[m] == x)
ans = m; r = m - 1; } else l = m + 1; } return ans; }

ll last_occurrence(const vll& a, ll l, ll r, ll x) { ll ans = -1; for(; l <= r; ) { ll m = (l + r) >> 1; if(a[m] <= x) { if(a[m] == x)
ans = m; l = m + 1; } else r = m - 1; } return ans; }

/* ---- YOO ---- */
int dx[4] = {1, -1, 0, 0};
int dy[4] = {0, 0, 1, -1};
void input(vector<ll>& v, ll n) { for(ll i = 0; i < n; i++) cin >> v[i]; }

//----PBDS----//
typedef tree<ll, null_type, less<ll>, rb_tree_tag, tree_order_statistics_node_update> ordered_set;
// find_by_order(k) -> iterator to kth (0-indexed)
// order_of_key(x) -> count of elements < x
void solve() {}

int main() {
    fastio;

```

```

    int t = 1;
    cin >> t;
    while (t--) solve();
    return 0;
}

```

SOME MORE:

```

//===== 🔥 BIT MANIPULATION =====//
/*

```

```

__builtin_popcount(x);    // count set bits (int)
__builtin_popcountll(x);  // count set bits (long long)

```

```

__builtin_clz(x);         // count leading zeros (int)
__builtin_clzll(x);       // count leading zeros (ll)

```

```

__builtin_ctz(x);         // count trailing zeros (int)
__builtin_ctzll(x);       // count trailing zeros (ll)

```

```

ll msb = 63 - __builtin_clzll(n); // index of most significant set bit
ll lsb = __builtin_ctzll(n);      // index of least significant set bit

```

```

if(n && !(n & (n-1)));          // check if power of 2

```

```

n |= (1LL << i); // set i-th bit
n &= ~(1LL << i); // unset i-th bit
n ^= (1LL << i); // toggle i-th bit

```

```

(n >> i) & 1; // check if i-th bit is set (0 or 1)

```

```

n &= (n - 1); // remove lowest set bit

```

```

n & (-n); // get lowest set bit value

```

```

ll x = stoll(s, nullptr, 2); // binary string → number
string s = bitset<32>(x).to_string(); // number → binary string

```

```

s = s.substr(s.find('1')); // remove leading zeros

```

```

//===== 🔥 SET =====//
set<ll> s;

```

```

s.insert(x);
s.erase(x);
s.count(x);
s.find(x);

```

```

*s.begin();
*s.rbegin();

```

```

s.lower_bound(x);
s.upper_bound(x);

```

```

s.size();
s.empty();

```

```

//===== 🔥 MULTiset =====//
multiset<ll> ms;

```

```

ms.insert(x);
ms.erase(ms.find(x));
ms.erase(x);

```

```

ms.count(x);

*ms.begin();
*ms.rbegin();

ms.lower_bound(x);
ms.upper_bound(x);

//===== 🔥 MAP (ORDERED) =====//
map<ll,ll> mp;

mp[x] = val;
mp.insert({x,val});

mp.erase(x);
mp.count(x);
mp.find(x);

mp.size();
mp.empty();

mp.lower_bound(x);
mp.upper_bound(x);

for(auto &[k,v]: mp){}

//===== 🔥 UNORDERED MAP =====//
unordered_map<ll,ll> ump;

ump[x] = val;
ump.erase(x);
ump.count(x);
ump.find(x);

ump.size();
ump.empty();

//===== 🔥 ORDERED SET (PBDS) =====//
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;

template<typename T>
using ordered_set = tree<T, null_type, less<T>, rb_tree_tag, tree_order_statistics_node_update>;

ordered_set<ll> os;

os.insert(x);
os.erase(x);

*os.find_by_order(k);
os.order_of_key(x);

//===== 🔥 PRIORITY QUEUE =====//
priority_queue<ll> pq;

pq.push(x);
pq.pop();
pq.top();
pq.size();
pq.empty();

```

```

priority_queue<ll, vector<ll>, greater<ll>> minpq;

priority_queue<pair<ll, ll>> pq2;
priority_queue<pair<ll, ll>, vector<pair<ll, ll>>, greater<pair<ll, ll>>> pq3;

//===== 🔥 VECTOR =====//
vector<ll> v;

v.push_back(x);
v.pop_back();

v.size();
v.empty();

v.front();
v.back();

sort(v.begin(), v.end());
sort(v.rbegin(), v.rend());

reverse(v.begin(), v.end());

*max_element(v.begin(), v.end());
*min_element(v.begin(), v.end());

accumulate(v.begin(), v.end(), 0LL);

//===== 🔥 DEQUE =====//
deque<ll> dq;

dq.push_back(x);
dq.push_front(x);

dq.pop_back();
dq.pop_front();

dq.front();
dq.back();

//===== 🔥 STACK =====//
stack<ll> st;

st.push(x);
st.pop();
st.top();
st.size();
st.empty();

//===== 🔥 QUEUE =====//
queue<ll> q;

q.push(x);
q.pop();
q.front();
q.back();
q.size();
q.empty();

//===== 🔥 ALGORITHMS =====//
sort(v.begin(), v.end());
sort(v.rbegin(), v.rend());

reverse(v.begin(), v.end());

```

```

binary_search(v.begin(), v.end(), x);

lower_bound(v.begin(), v.end(), x);
upper_bound(v.begin(), v.end(), x);

next_permutation(v.begin(), v.end());
prev_permutation(v.begin(), v.end());

__gcd(a,b);
lcm(a,b);

//===== 🔥 PAIRS =====//
pair<ll,ll> p;

p.first;
p.second;

make_pair(a,b);

sort(v.begin(), v.end()); // works for vector<pair>

//===== 🔥 STRING =====//
s.size();
s.substr(l, len);

stoi(s);
stoll(s);

to_string(x);

reverse(s.begin(), s.end());

//===== 🔥 CUSTOM COMPARATOR =====//
//----- ♦ SORT (vector / array) -----//

// ascending by default
sort(v.begin(), v.end());

// descending
sort(v.begin(), v.end(), greater<ll>());

// custom comparator (function)
bool cmp(ll a, ll b){
    return a > b; // descending
}
sort(v.begin(), v.end(), cmp);

// custom comparator (lambda)
sort(v.begin(), v.end(), [](ll a, ll b){
    return a > b;
});

// pair sorting
vector<pair<ll,ll>> v;

// sort by first ascending, second descending
sort(v.begin(), v.end(), [](pll a, pll b){
    if(a.first == b.first) return a.second > b.second;
    return a.first < b.first;
});

```

```
//----- ♦ SET / MAP CUSTOM ORDER -----//
```

```
// descending set  
set<ll, greater<ll>> s;
```

```
// custom comparator struct  
struct cmpSet{  
    bool operator()(ll a, ll b) const{  
        return a > b; // descending  
    }  
};  
set<ll, cmpSet> s2;
```

```
//----- ♦ PRIORITY QUEUE -----//
```

```
// max heap (default)  
priority_queue<ll> pq;
```

```
// min heap  
priority_queue<ll, vector<ll>, greater<ll>> pq2;
```

```
// custom comparator (pair)  
priority_queue<pll, vector<pll>, greater<pll>> pq3;
```

```
// custom struct comparator  
struct cmpPQ{  
    bool operator()(pll a, pll b){  
        return a.second > b.second; // min by second  
    }  
};  
priority_queue<pll, vector<pll>, cmpPQ> pq4;
```

```
//----- ♦ MULTISSET CUSTOM -----//
```

```
multiset<ll, greater<ll>> ms;
```

```
//----- ♦ IMPORTANT RULE -----//
```

```
// comparator should return TRUE if a should come BEFORE b  
// i.e., defines strict ordering
```

```
// WRONG ❌ (can cause TLE / crash)  
// return a <= b;
```

```
// CORRECT ✅  
// return a < b;  
*/
```

MORE FUNCTIONS;

```
//===== 🔥 1. BINARY SEARCH TEMPLATES =====//  
/*
```

```
// first index i such that v[i] >= x  
ll first_ge(vll &v, ll x){  
    ll l=0, r=v.size()-1, ans=v.size();  
    while(l<=r){  
        ll m=(l+r)/2;
```

```

        if(v[m]>=x) ans=m,r=m-1;
        else l=m+1;
    }
    return ans;
}

```

```

// first index i such that v[i] > x
ll first_gt(vll &v, ll x){
    ll l=0,r=v.size()-1,ans=v.size();
    while(l<=r){
        ll m=(l+r)/2;
        if(v[m]>x) ans=m,r=m-1;
        else l=m+1;
    }
    return ans;
}

```

```

// last index i such that v[i] <= x
ll last_le(vll &v, ll x){
    ll l=0,r=v.size()-1,ans=-1;
    while(l<=r){
        ll m=(l+r)/2;
        if(v[m]<=x) ans=m,l=m+1;
        else r=m-1;
    }
    return ans;
}

```

```

// last index i such that v[i] < x
ll last_lt(vll &v, ll x){
    ll l=0,r=v.size()-1,ans=-1;
    while(l<=r){
        ll m=(l+r)/2;
        if(v[m]<x) ans=m,l=m+1;
        else r=m-1;
    }
    return ans;
}

```

```

// binary search on answer
ll bs_answer(ll l, ll r){
    while(l<=r){
        ll m=(l+r)/2;
        if(check(m)) r=m-1;
        else l=m+1;
    }
    return l;
}

```

//===== 🔥 2. DP TEMPLATES =====//

```

// memoization
ll dp[N];
ll f(ll i){
    if(i==0) return 1;
    if(dp[i]!=-1) return dp[i];
    return dp[i]=f(i-1);
}

```

```

// knapsack
for(ll i=0;i<n;i++){
    for(ll j=W;j>=wt[i];j--){
        dp[j]=max(dp[j],dp[j-wt[i]]+val[i]);
    }
}

```



```

    }
}

//----1. Bitmask DP (TSP / minimum path visiting all nodes)----//
ll dp_mask[1<<20][20];
ll tsp(ll mask,ll u,ll n,vector<vll>&dist){
    if(mask==(1<<n)-1) return 0;
    ll &res=dp_mask[mask][u];
    if(res!=-1) return res;
    res=1e18;
    for(ll v=0;v<n;v++){
        if(!(mask&(1<<v))){
            res=min(res,dist[u][v]+tsp(mask|(1<<v),v,n,dist));
        }
    }
    return res;
}

//----2. LCS (Longest Common Subsequence)----//
ll lcs(string &a,string &b){
    ll n=a.size(),m=b.size();
    vector<vector<ll>>dp(n+1,vector<ll>(m+1,0));
    for(ll i=1;i<=n;i++){
        for(ll j=1;j<=m;j++){
            if(a[i-1]==b[j-1]) dp[i][j]=1+dp[i-1][j-1];
            else dp[i][j]=max(dp[i-1][j],dp[i][j-1]);
        }
    }
    return dp[n][m];
}

//----3. Palindrome DP (is substring i..j palindrome)----//
vector<vector<bool>> isPal;
void build_pal(string &s){
    ll n=s.size();
    isPal.assign(n,vector<bool>(n,0));
    for(ll i=0;i<n;i++) isPal[i][i]=1;
    for(ll len=2;len<=n;len++){
        for(ll i=0;i+len-1<n;i++){
            ll j=i+len-1;
            if(s[i]==s[j]&&(len==2 || isPal[i+1][j-1]))
                isPal[i][j]=1;
        }
    }
}

//----4. Longest Palindromic Substring (using DP)----//
ll longest_palindrome(string s){
    ll n=s.size(),ans=1;
    vector<vector<bool>>dp(n,vector<bool>(n,0));
    for(ll i=0;i<n;i++) dp[i][i]=1;
    for(ll len=2;len<=n;len++){
        for(ll i=0;i+len-1<n;i++){
            ll j=i+len-1;
            if(s[i]==s[j]&&(len==2 || dp[i+1][j-1])){
                dp[i][j]=1;
                ans=max(ans,len);
            }
        }
    }
    return ans;
}

```

```

//----5. House Robber (max sum no adjacent)----//
ll house_robber(vll &nums){
    ll n=nums.size();
    if(n==0) return 0;
    if(n==1) return nums[0];
    vll dp(n);
    dp[0]=nums[0];
    dp[1]=max(nums[0],nums[1]);
    for(ll i=2;i<n;i++){
        dp[i]=max(dp[i-1],dp[i-2]+nums[i]);
    }
    return dp[n-1];
}

//===== 🔥 3. GRAPH TEMPLATES =====//

// adjacency list
vector<vll> adj(n+1);

// DFS
void dfs(ll u, ll p){
    for(auto v:adj[u]){
        if(v==p) continue;
        dfs(v,u);
    }
}

// BFS
void bfs(ll src){
    queue<ll> q;
    vector<ll> vis(n+1,0);
    q.push(src); vis[src]=1;
    while(!q.empty()){
        ll u=q.front(); q.pop();
        for(auto v:adj[u]){
            if(!vis[v]){
                vis[v]=1;
                q.push(v);
            }
        }
    }
}

// Dijkstra
vector<ll> dijkstra(ll src){
    vll dist(n+1,1e18);
    priority_queue<pll,vector<pll>,greater<pll>> pq;
    dist[src]=0;
    pq.push({0,src});
    while(!pq.empty()){
        auto [d,u]=pq.top(); pq.pop();
        if(d>dist[u]) continue;
        for(auto [v,w]:adj[u]){
            if(dist[v]>d+w){
                dist[v]=d+w;
                pq.push({dist[v],v});
            }
        }
    }
    return dist;
}

//-----DSU + APPLICATIONS-----//

```

```

vll parent,sz;

//find with path compression
ll find(ll x){return parent[x]==x?x:parent[x]=find(parent[x]);}

//union by size
void unite(ll a,ll b){
    a=find(a);b=find(b);
    if(a!=b){
        if(sz[a]<sz[b]) swap(a,b);
        parent[b]=a;
        sz[a]+=sz[b];
    }
}

//init DSU
void init_dsu(ll n){
    parent.resize(n+1);
    sz.assign(n+1,1);
    iota(all(parent),0);
}

//----1. Cycle Detection (Undirected Graph)----//
bool hasCycle(ll n,vector<pair<ll,ll>>&edges){
    init_dsu(n);
    for(auto [u,v]:edges){
        if(find(u)==find(v)) return 1;
        unite(u,v);
    }
    return 0;
}

//----2. Number of Connected Components----//
ll components(ll n){
    set<ll>s;
    for(ll i=1;i<=n;i++) s.insert(find(i));
    return s.size();
}

//----3. Kruskal MST----//
ll kruskal(ll n,vector<tuple<ll,ll,ll>>&edges){
    init_dsu(n);
    sort(all(edges)); // sort by weight
    ll cost=0;
    for(auto [w,u,v]:edges){
        if(find(u)!=find(v)){
            unite(u,v);
            cost+=w;
        }
    }
    return cost;
}

//----4. Check if Graph is Tree----//
bool isTree(ll n,vector<pair<ll,ll>>&edges){
    if(edges.size()!=n-1) return 0;
    return !hasCycle(n,edges);
}

//----5. Component Sizes----//
vll comp_size(ll n){
    vll res(n+1);
    for(ll i=1;i<=n;i++) res[find(i)]++;
}

```

```

    return res;
}

//----6. Dynamic Connectivity Queries----//
// type 1 u v -> unite
// type 2 u v -> check connected
void dynamic_queries(){
    ll q;cin>>q;
    while(q--){
        ll t,u,v;cin>>t>>u>>v;
        if(t==1) unite(u,v);
        else cout<<(find(u)==find(v))<<endl;
    }
}

//----7. Number of Islands (Grid DSU)----//
ll islands(vector<vector<int>>&g){
    ll n=g.size(),m=g[0].size();
    init_dsu(n*m);
    auto id=[&](ll x,ll y){return x*m+y;};
    for(ll i=0;i<n;i++)for(ll j=0;j<m;j++){
        if(!g[i][j]) continue;
        for(ll k=0;k<4;k++){
            ll ni=i+(ll[]){1,-1,0,0}[k],nj=j+(ll[]){0,0,1,-1}[k];
            if(ni>=0&&nj>=0&&ni<n&&nj<m&&g[ni][nj])
                unite(id(i,j),id(ni,nj));
        }
    }
    set<ll>s;
    for(ll i=0;i<n;i++)for(ll j=0;j<m;j++){
        if(g[i][j]) s.insert(find(id(i,j)));
    }
    return s.size();
}

//----8. DSU with Rollback (Advanced)----//
stack<pair<ll,ll>> st;

ll find_rb(ll x){while(x!=parent[x]) x=parent[x]; return x;}

void unite_rb(ll a,ll b){
    a=find_rb(a);b=find_rb(b);
    if(a==b){st.push({-1,-1});return;}
    if(sz[a]<sz[b]) swap(a,b);
    parent[b]=a;sz[a]+=sz[b];
    st.push({a,b});
}

void rollback(){
    auto [a,b]=st.top();st.pop();
    if(a==-1) return;
    parent[b]=b;
    sz[a]-=sz[b];
}

//===== 🔥 4. IMPORTANT ALGORITHMS =====//

// Kadane
ll kadane(vll &a){
    ll sum=0,ans=-1e18;
    for(auto x:a){
        sum=max(x,sum+x);
        ans=max(ans,sum);
    }
}

```

```

    return ans;
}

// prefix sum
vll pref(n+1,0);
for(ll i=1;i<=n;i++) pref[i]=pref[i-1]+a[i];

//-----SPF (Smallest Prime Factor)-----//

const int MAXN=200005;
int spf[MAXN];

//build sieve
void sieve(){
    iota(spf,spf+MAXN,0);
    for(int i=2;i*i<MAXN;i++)
        if(spf[i]==i)
            for(int j=i*i;j<MAXN;j+=i)
                if(spf[j]==j) spf[j]=i;
}

//----1. check prime O(1)----//
bool isPrime(ll x){return spf[x]==x;}

//----2. unique prime factors----//
vll get_fac(ll x){
    vll f;
    while(x>1){
        ll p=spf[x];
        f.pb(p);
        while(x%p==0) x/=p;
    }
    return f;
}

//----ALL divisors in logN----//
vll getDivisors(ll x){
    vll primes, cnt;
    while(x>1){
        ll p=spf[x],c=0;
        while(x%p==0){x/=p;c++;}
        primes.pb(p);
        cnt.pb(c);
    }

    vll divs={1};
    for(int i=0;i<primes.size();i++){
        vll temp;
        ll cur=1;
        for(int j=1;j<=cnt[i];j++){
            cur*=primes[i];
            for(auto d:divs) temp.pb(d*cur);
        }
        for(auto v:temp) divs.pb(v);
    }
    return divs;
}

//----3. prime factor count (with frequency)----//
map<ll,ll> get_fac_cnt(ll x){
    map<ll,ll> mp;
    while(x>1){
        mp[spf[x]]++;
    }
}

```

```

        x/=spf[x];
    }
    return mp;
}

//----4. number of divisors----//
ll num_div(ll x){
    ll ans=1;
    while(x>1){
        ll p=spf[x],cnt=0;
        while(x%p==0){cnt++;x/=p;}
        ans*=(cnt+1);
    }
    return ans;
}

//----5. sum of divisors----//
ll sum_div(ll x){
    ll ans=1;
    while(x>1){
        ll p=spf[x],cur=1,sum=1;
        while(x%p==0){
            x/=p;
            cur*=p;
            sum+=cur;
        }
        ans*=sum;
    }
    return ans;
}

// gcd / lcm

ll gcd(ll a,ll b){ return b==0?a:gcd(b,a%b); }
ll lcm(ll a,ll b){ return a/gcd(a,b)*b; }

*/

```